



# Loops & Problem Solving in JavaScript

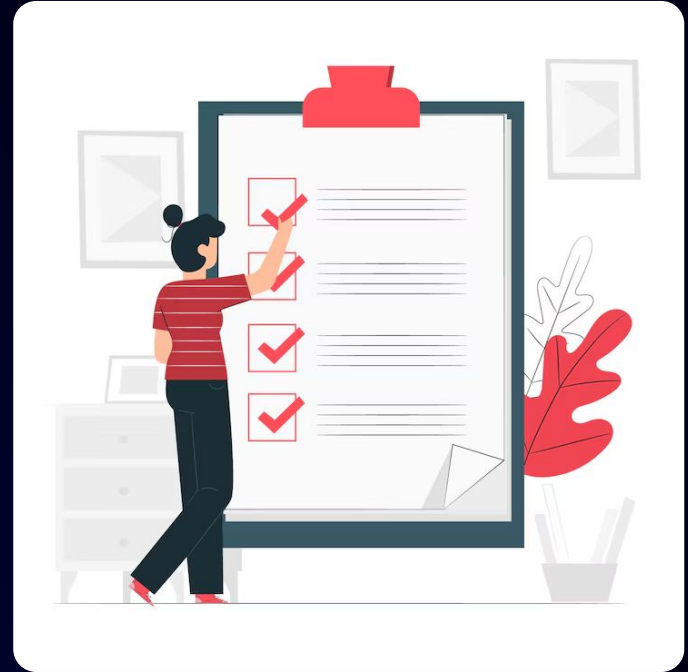
# Learning Objectives

- Use loops (for, while, do...while) for repetition
- Apply break and continue for control flow
- Understand and use switch statements
- Choose between switch and if-else
- Solve problems using loops and control statements



# Prerequisites

- Basic understanding of programming concepts
- Familiarity with variables, data types, and conditions
- Basic knowledge of loops (for, while)
- Understanding of simple decision-making (if-else)
- Familiarity with writing and running code in an editor/IDE
- Basic debugging and problem-solving skills
- Willingness to practice and experiment with logic building





# Why Loops are Needed

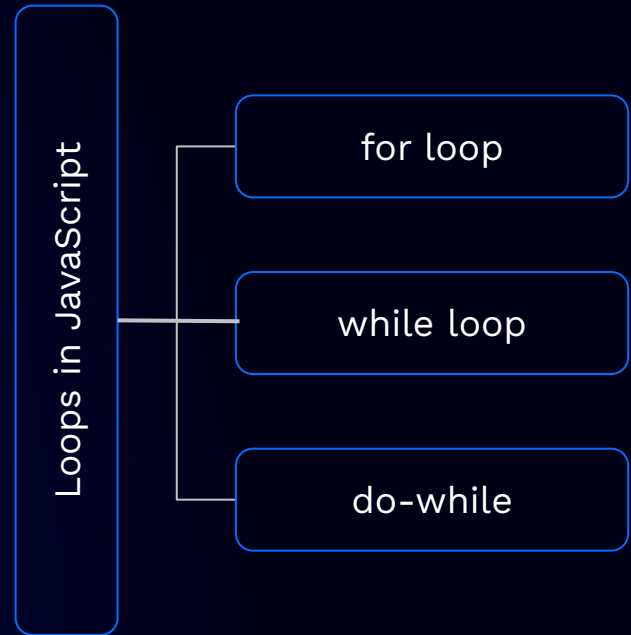
# Overview of Loops

- Loops, or iteration statements, are essential in programming and critical to modern computing.
- They enable automation by allowing code to run repeatedly based on specific instructions.
- A loop is a block of code with instructions on how long to keep it running repeatedly.

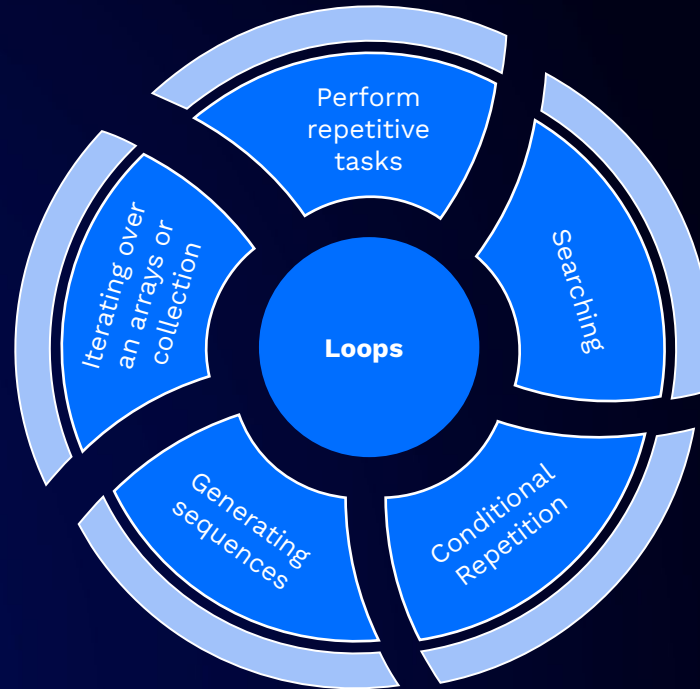
**JS** Loops

# Types of loops in JavaScript

- The basic types of loops in JavaScript are
  - for loop
  - while loop
  - do-while loop



# Usage of loops



# Pop Quiz

Q. Select the basic types of loop in javascript

**A**

**do while**

**B**

**If else**



# Pop Quiz

Q. Select the basic types of loop in javascript

**A**

**do while**

**B**

**If else**



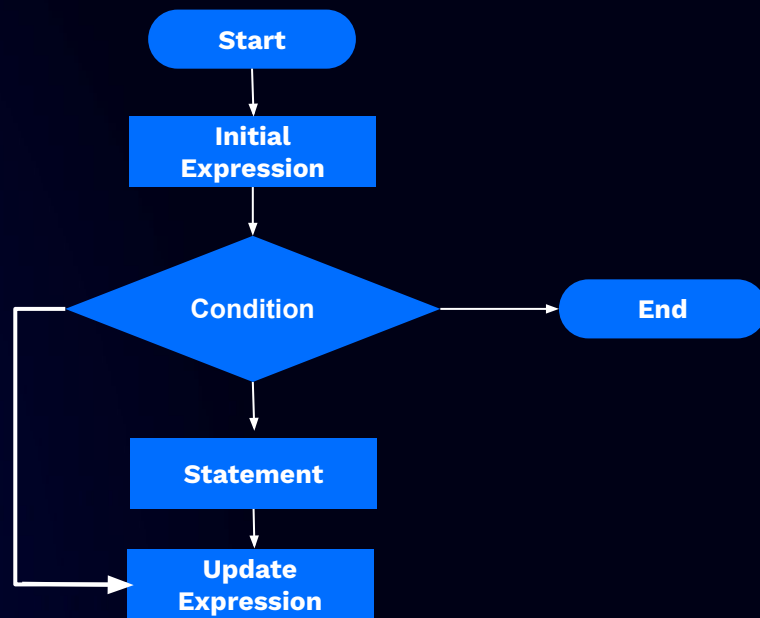
# for loop

# For loop in JavaScript

- The **for** loop is meant to repeatedly execute a piece of code a known number of times.
- We use **for** loop to run a piece of code until the set condition turns false
- Syntax -

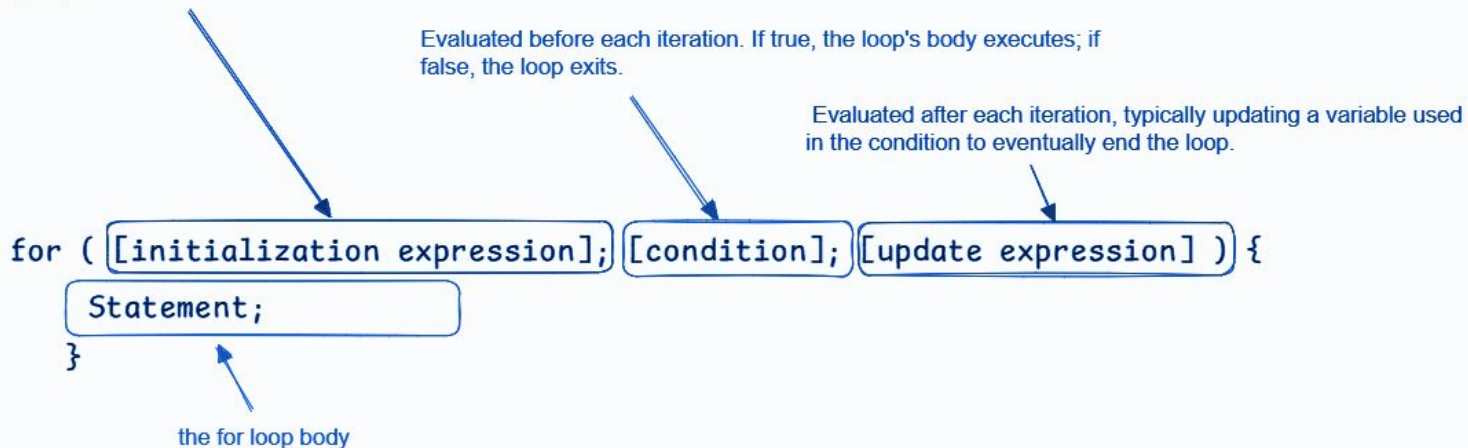
```
for ([initialization expression]; [condition]; [update expression]){  
    Statement;  
}
```

## Flow of the for loop



# Understanding the for loop syntax

Variables used in the loop are initialized, and can also be declared (e.g., using var).





## Hands-On Demo | for loop in JavaScript

# Pop Quiz

**Q. Which statement is true about the for loop's update expression?**

**A**

**It is executed after the loop's  
body**

**B**

**It is executed before the  
loop's body**

# Pop Quiz

**Q. Which statement is true about the for loop's update expression?**

**A**

**It is executed after the loop's  
body**

**B**

**It is executed before the  
loop's body**



# while loop

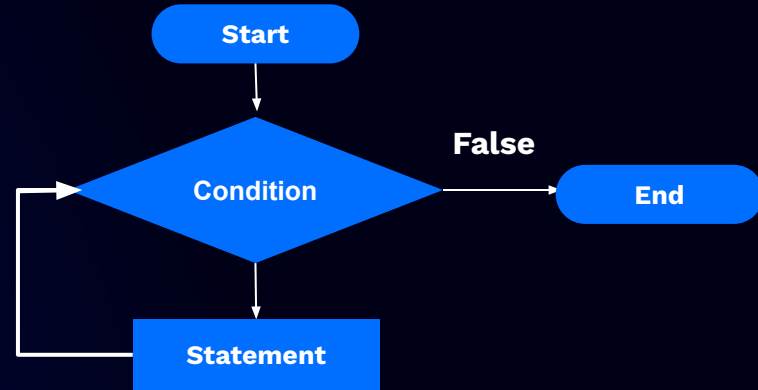


# while loop in JavaScript

- The while loop is meant to repeatedly execute a piece of code an unknown number of times.
- This loop keeps running as long as the condition is true

## Syntax -

```
while (condition) {  
    // body of the loop  
}
```



Flow of the while loop



## Hands-On Demo | while loop in JavaScript

# Pop Quiz

Q. What will be the output of the following code ?

```
let i = 0;  
while (i < 3) {  
  console.log(i);  
  i++;  
}
```

A

0 1 2 3

B

0 1 2

# Pop Quiz

Q. What will be the output of the following code ?

```
let i = 0;  
while (i < 3) {  
  console.log(i);  
  i++;  
}
```

A

0 1 2 3

B

0 1 2



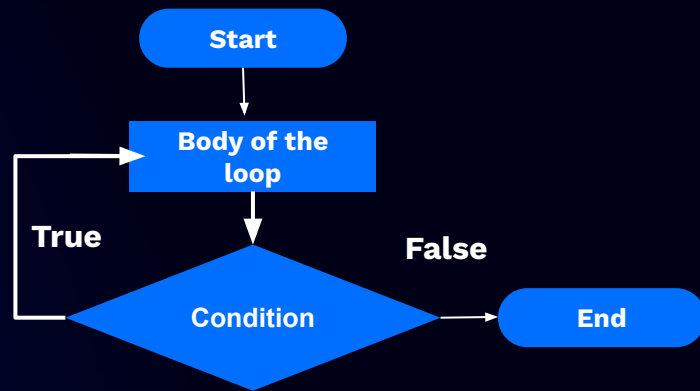
# do-while loop

# do-while loop in JavaScript

- Do while loop is similar to while loop with one difference that the 1st iteration runs always, and later iterations run after the condition evaluates to true
- Syntax -

## Syntax -

```
do {  
    // body of the loop  
} while (condition)
```



Flow of the do-while loop

# Comparison on for, while and do while loops

| Feature   | for Loop                                                               | while Loop                                                       | do-while Loop                                    |
|-----------|------------------------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------|
| Syntax    | <pre>for (initialization;<br/>condition; update)<br/>{ // code }</pre> | <pre>while (condition) { // code<br/>}</pre>                     | <pre>do { // code } while<br/>(condition);</pre> |
| condition | Before each iteration                                                  | Before each iteration                                            | After each iteration                             |
| Execution | Not guaranteed (may not execute if condition is false initially)       | Not guaranteed (may not execute if condition is false initially) | Guaranteed to execute at least once              |

# Comparison on for, while and do while loops

| Feature   | for Loop                                                           | while Loop                                                                                           | do...while Loop                                                               |
|-----------|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Use cases | When the number of iterations is known or can be easily determined | When the number of iterations is not known, and loop should execute as long as the condition is true | When the loop must execute at least once regardless of the condition          |
| condition | Before each iteration                                              | Before each iteration                                                                                | After each iteration                                                          |
| Example   | <pre>for (let i = 0; i &lt; 5; i++)<br/>{ console.log(i); }</pre>  | <pre>let i = 0; while (i &lt; 5) {<br/>  console.log(i); i++; }</pre>                                | <pre>let i = 0; do {<br/>  console.log(i); i++; }<br/>while (i &lt; 5);</pre> |





## Hands-On Demo | do-while loop

# Pop Quiz

Q. What will be the output of the following code ?

```
let count = 0;  
do {  
    count++;  
} while (count < 0);  
console.log(count);
```

A

1

B

0

# Pop Quiz

Q. What will be the output of the following code ?

```
let count = 0;  
do {  
    count++;  
} while (count < 0);  
console.log(count);
```

A

1

B

0

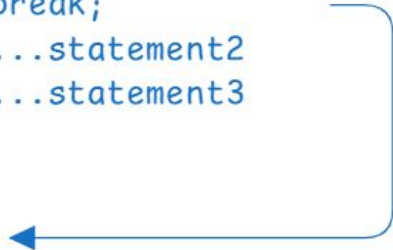


# Break and Continue

# Break statement in JavaScript

- break statement, without a label reference, can only be used to jump out of a loop or a switch block.
- It terminates the current loop or switch statement and transfers program control to the statement following the terminated statement.
- **Syntax -**  
**break;**

```
while (condition) {  
    ...statement0  
    ...statement1  
    break;  
    ...statement2  
    ...statement3  
}
```



# Continue statement in JavaScript

- **continue** statement, with or without a label reference, can only be used to skip one loop iteration.
- Skips the execution of the statements in the current iteration of the current or labeled loop, and continues execution of the loop with the next iteration.
- **Syntax -**  
**continue;**

```
while (condition) {  
    ...statement0  
    ...statement1  
    continue;  
    ...statement2  
    ...statement3  
}
```



# Comparing break and continue statements

| Statement | Purpose                                                                                              | Usage Example                                                                                                          | Typical use case                                       |
|-----------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| continue  | Skips the rest of the code inside the loop for the current iteration and moves to the next iteration | <pre>javascript for (let i = 0; i &lt; 5; i++) { if (i === 2) continue; console.log(i); }<br/>// Output: 0 1 3 4</pre> | When you need to skip certain iterations in a loop     |
| break     | Exits the loop immediately, and the control moves to the code following the loop                     | <pre>javascript for (let i = 0; i &lt; 5; i++) { if (i === 2) break; console.log(i); }<br/>// Output: 0 1</pre>        | When you need to terminate a loop based on a condition |



## **Hands-On Demo | Break and Continue Statements**



# Pop Quiz

Q. What will be the output of the following code ?

```
for (let i = 0; i < 5; i++) {  
    if (i === 3) break;  
    if (i === 1) continue;  
    console.log(i);  
}
```

A

0 1 2

B

0 2

# Pop Quiz

Q. What will be the output of the following code ?

```
for (let i = 0; i < 5; i++) {  
  if (i === 3) break;  
  if (i === 1) continue;  
  console.log(i);  
}
```

A

0 1 2

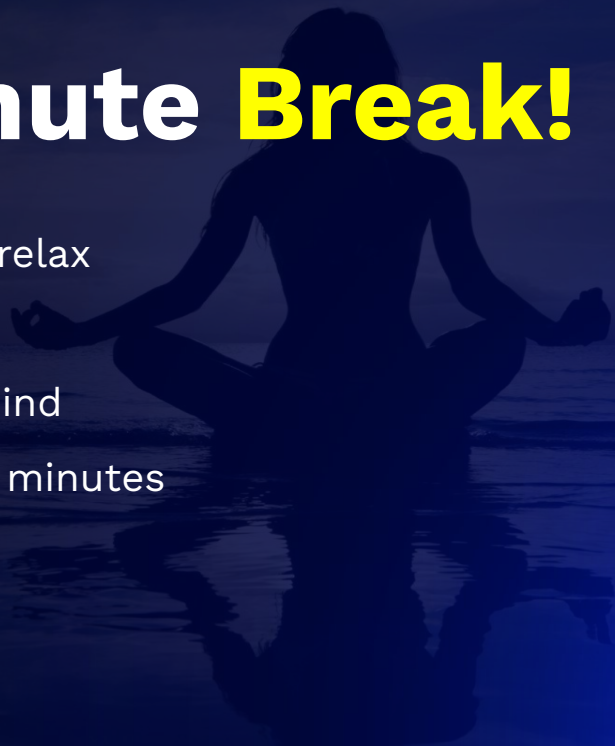
B

0 2



# Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes





# Switch Deep Dive

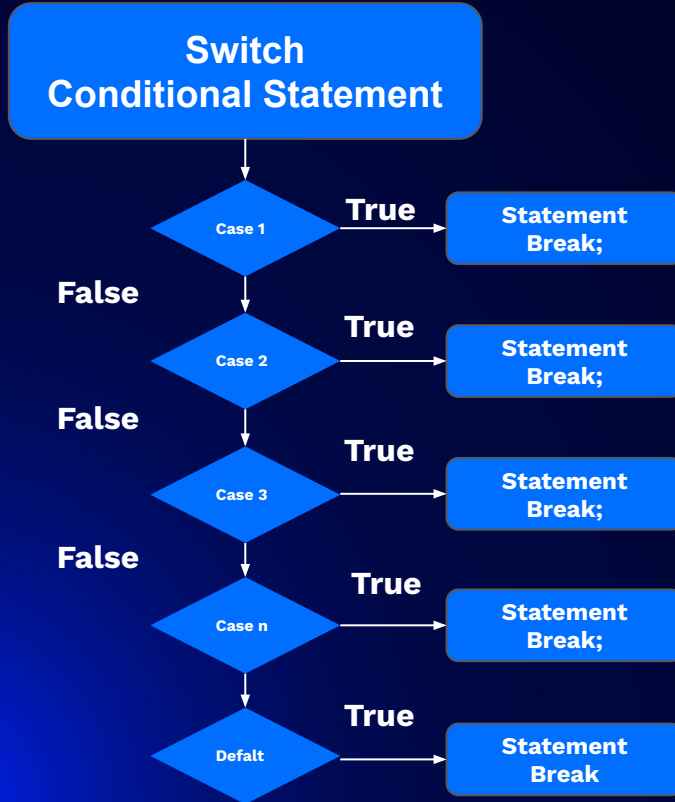
# Introduction to Switch statement and Syntax

- The switch statement evaluates an expression.
- It matches the expression's value against case clauses.
- Executes statements after the first matching case until a break statement is encountered.
- The default clause executes if no case matches the expression's value.

```
switch (expression) {  
  case value1:  
    //Statements executed when the result of expression matches value1  
    break;  
  case value2:  
    //Statements executed when the result of expression matches value2  
    break;  
  ...  
  case valueN:  
    //Statements executed when the result of expression matches valueN  
    break;  
  default:  
    //Statements executed when none of the values match the value of the expression  
    break;  
}
```

**Syntax of the switch statement**

# Flow of the Switch statement



# Some of the usage cases of Switch statement

- When you need to perform different actions based on user input or choices.
- Executing different code based on the day of the week.
- Managing application states or modes.
- Managing access to different parts of an application based on user roles.
- Handling different game states or player actions.

# Comparison of the Switch and if statements

| Feature               | Switch Statement                                                         | if Statement                                                  |
|-----------------------|--------------------------------------------------------------------------|---------------------------------------------------------------|
| Purpose               | Best for comparing a single expression against multiple possible values. | Best for evaluating multiple different or complex conditions. |
| Syntax Readability    | More readable for simple value comparisons.                              | More flexible for complex logical conditions.                 |
| Fall-Through Behavior | Executes all cases after a matching one unless break is used.            | Executes only the matched block; no fall-through.             |



# Comparison of the Switch and if statements

| Feature          | Switch Statement                                          | if Statement                                                         |
|------------------|-----------------------------------------------------------|----------------------------------------------------------------------|
| Performance      | Generally faster for multiple discrete value comparisons. | Potentially slower with many conditions due to multiple evaluations. |
| Default Handling | Uses default clause for unmatched cases.                  | Uses else clause for unmatched conditions.                           |



## Hands-On Demo | Switch Statements

# Pop Quiz

Q. What will be the output of the following code ?

```
let text = 'JavaScript';  
switch (text[1]) {  
  case 'a':  
  case 'e':  
  case 'i':  
  case 'o':  
  case 'u':  
    console.log('Vowel');  
  default:  
    console.log('Consonant');  
}
```

A

Vowel  
Consonant

B

vowel

# Pop Quiz

Q. What will be the output of the following code ?

```
let text = 'JavaScript';  
switch (text[1]) {  
  case 'a':  
  case 'e':  
  case 'i':  
  case 'o':  
  case 'u':  
    console.log('Vowel');  
  default:  
    console.log('Consonant');  
}
```

A

Vowel  
Consonant

B

vowel



# Logical Problem Solving Patterns

# Collaboration Workflow Using Branches

In team projects, multiple developers work together 

Best practice:

-  Do NOT work directly on main
-  Use separate branches for features

Helps:

- Avoid conflicts
- Keep main branch stable

Commonly used on GitHub

# Pop Quiz

Q. What is a limitation of using GUI tools like GitHub Desktop?

**A**

**Deep understanding of Git  
concept**

**B**

**Less understanding of Git  
concepts**



## Practical Case Studies (Hands-On)

- print even numbers from 1 to n
- sum of numbers from 1 to n
- count numbers divisible by 3
- simple menu using switch



# Summary

In this lecture, we have covered:

- Understand the need for loops in programming
- Use for, while, and do...while loops effectively
- Apply break and continue for control flow
- Use switch for decision-making
- Solve basic logical problems using loops
- Implement practical programs (number printing, sum, divisibility, menu systems)

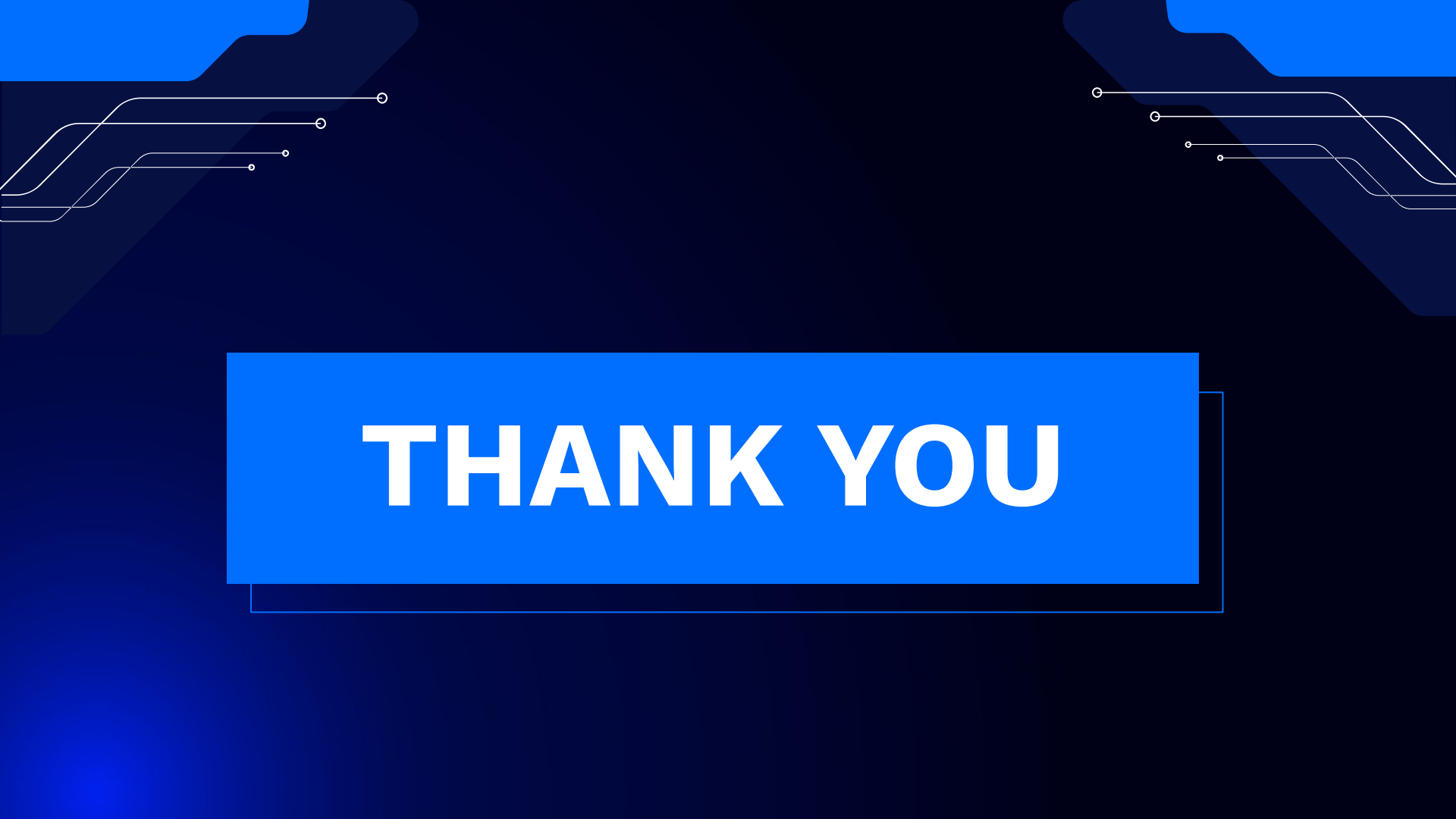




# DOUBT RESOLUTION

# Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



**THANK YOU**